

SAÉ 3.B.01

FEN-Alim

Tests réalisés

Valentin HUTA-CEVAER GR2B || Clovis BOURRE GR2B || Mathéo BIET GR2B
|| Tei GARNIER GR2C || Clément HOARAU GR2C



Tests du DAO :	4
Module Utilisateur :	4
Justification :	4
Illustrations avec les tests principaux :	4
Test unitaire : logique de hachage	4
Test fonctionnels : Intégrité du profil	4
Test intégration : Contraintes de la base de données	5
Test sécurité : Vérification d'accès	6
Module Point d'eau :	6
Justification :	6
Illustration avec les tests principaux :	6
Test unitaire : transformation de données	6
Test fonctionnel : Protection des champs critiques	7
Test d'intégration : flux CRUD complet	8
Test sécurité : Unicité du PEI	9
Modules Missions :	9
Justification :	9
Illustration avec les tests principaux :	10
Test unitaire : Transformation et accès	10
Test fonctionnel : Filtrage métier	10
Test d'intégration : Cycle de vie complet	10
Test sécurité : Validation des clés étrangères	11
Module Historique :	11
Justification :	12
Illustration avec les tests principaux :	12
Test unitaire : intégrité des types de retour	12
Test fonctionnel : Tri chronologique	12
Test d'intégration : Flux d'activité	13
Test sécurité : Résistance aux injections SQL	13
Module Signaler :	14
Justification :	14
Illustration avec les tests principaux :	14
Test unitaire : validation du schéma de sortie	14
Test fonctionnel : Accumulation de signalement	15
Test d'intégrité : Nettoyage du stockage (Photo)	16
Test sécurité : Intégrité référentielle	17
Test des routes API	17
Module Utilisateurs :	17
Justification :	17
Illustration avec les tests principaux :	18
Test d'Authentification : Création et Token	18
Test de Profil : Injection de Dépendance	18
Test de Sécurité : Contrôle des Rôles (RBAC)	19
Test de Protection : Auto-suppression interdite	19

Routes Point d'eau :	19
Justification :	19
Illustration avec les tests principaux :	20
Test de Consultation : Récupération par Numéro PEI	20
Test de Création : Validation et Géolocalisation	20
Test de Sécurité : Protection des données	20
Test de Suppression : Nettoyage administratif	21
Routes Missions :	21
Justification :	21
Illustration avec les tests principaux :	22
Test de Création : Attribution de tâche	22
Test de Mise à Jour : Clôture de Mission	22
Test de Consultation : Liste de travail	23
Test de Suppression : Sécurité administrative	23
Routes Historique :	24
Justification :	24
Illustration avec les tests principaux :	24
Test de Sécurité : Accès Interdit au Public	24
Test de Consultation : Audit d'un Utilisateur	24
Test de Performance : Limitation des résultats	25
Test de Privilège : Création d'Audit	25
Route Signaler :	26
Justification :	26
Illustration avec les tests principaux :	26
Test de Création : Upload de photo et Signalement	26
Test de Gestion d'Erreur : Point Inexistant	27
Test de Consultation : Flux d'incidents	28
Test de Suppression : Sécurité Admin	28
Conclusion	29

Tests du DAO :

Le DAO est le moteur de notre application. Le choix de tester cette couche repose sur une nécessité de stabilité : si l'accès à la base de données est défaillant, toute l'application s'effondre :

Module Utilisateur :

Pour le module Utilisateurs, nous avons priorisé la validation des processus d'inscription, de sécurité des données sensibles et de gestion des droits de modification.

Justification :

- Tests Unitaires : Ils isolent les fonctions de logique pure (comme le hachage). Il est crucial de garantir que le mot de passe ne circule jamais en clair avant même d'atteindre la base de données.
- Tests Fonctionnels : Ils valident que les règles métier sont respectées (ex: un utilisateur ne peut pas changer son propre rôle admin via son profil).
- Tests d'Intégration : Ils vérifient la communication réelle avec la base de données PostgreSQL, notamment le respect des contraintes d'unicité (email et téléphone).
- Tests de sécurité : Ils se concentrent sur la protection des identifiants et la vérification de l'imperméabilité des mots de passe hachés.

Illustrations avec les tests principaux :

Test unitaire : logique de hachage

Ce test vérifie de manière isolé que l'algorithme transforme bien du texte brut en une empreinte sécurisé

```
def test_hash_password(self):  
    password = "TestPassword123"  
    hashed = hash_password(password)  
    assert hashed != password  
    assert len(hashed) > 0
```

Test fonctionnels : Intégrité du profil

Ici, on s'assure qu'une fonction métier (`update_own_profile`) respecte les restrictions de sécurité : un utilisateur peut modifier son nom, mais pas son rôle.

```
def test_update_own_profile_only_allowed_fields(self, db_session,
sample_user_data):
    user = create_utilisateur(db_session, sample_user_data)

    update_data = {
        "nom": "Nouveau",
        "mot_de_passe": "HackedPassword",
        "role": "admin"
    }
    updated = update_own_profile(db_session, user.id_utilisateur,
update_data)

    assert updated.nom == "Nouveau"
    assert updated.role == "public"
    assert verify_password("Password123!", updated.mot_de_passe)
```

Test intégration : Contraintes de la base de données

Ce test vérifie que la base de données rejette correctement les doublons de numéros de téléphone, garantissant l'intégrité des données réelles.

```
def test_update_own_profile_duplicate_telephone(self, db_session,
sample_user_data):
    user1 = create_utilisateur(db_session, sample_user_data)

    user2_data = sample_user_data.copy()
    user2_data["email"] = "second@example.com"
    user2_data["telephone"] = "0707070707"
    user2 = create_utilisateur(db_session, user2_data)

    with pytest.raises(ValueError, match="Numéro de téléphone déjà
utilisé"):
        update_own_profile(db_session, user2.id_utilisateur,
{"telephone": user1.telephone})
```

Test sécurité : Vérification d'accès

Ce test valide que la fonction de vérification distingue correctement un mot de passe valide d'une tentative d'intrusion.

```
def test_verify_user_password_incorrect(self, db_session,
sample_user_data):
    user = create_utilisateur(db_session, sample_user_data)
    result = verify_user_password(db_session, user.id_utilisateur,
"WrongPassword")
    assert result is False
```

Module Point d'eau :

Ici les tests DAO ici ne se limitent pas à la lecture/écriture, ils garantissent la précision géographique et la conformité des données techniques.

Justification :

- Tests unitaires : Essentiels pour valider les transformations automatiques, comme la conversion des statuts en majuscules ou le calcul des coordonnées géométriques Lambert93 à partir de la latitude/longitude.
- Tests fonctionnels : Ils simulent des cas métiers, comme la mise à jour partielle d'un point d'eau tout en vérifiant que les champs protégés (ID, date de création) restent immuables.
- Tests d'Intégration : Ils valident le cycle de vie complet d'un PEI, de sa création à sa suppression, en s'assurant que les recherches par numero_pei (identifiant métier) et par id (identifiant technique) sont cohérentes.
- Tests de sécurité: Ils préviennent l'injection de doublons. Dans le contexte incendie, deux bornes ne peuvent pas avoir le même numéro PEI sans compromettre la sécurité des opérations.

Illustration avec les tests principaux :

Test unitaire : transformation de données

Ce test montre que le DAO normalise les données entrantes. Si un utilisateur saisit un statut en minuscules, le système doit le convertir pour respecter les standards de la base de données.

```
def test_create_point_eau_uppercase_conversion(self, db_session):
    """Test que statut et type_nature sont convertis en
    majuscules"""
    data = {
        "numero_pei": 77777,
        "statut": "public",
        "type_nature": "bi",
        "latitude": 48.0,
        "longitude": 2.0
    }

    point = create_point_eau(db_session, data)
    assert point.statut == "PUBLIC"
    assert point.type_nature == "BI"
```

Test fonctionnel : Protection des champs critiques

Ce test garantit l'intégrité historique des données en vérifiant qu'une mise à jour ne peut pas modifier accidentellement la date de création ou l'identifiant unique.

```
def test_update_point_eau_ignore_protected_fields(self, db_session,
sample_point):
    original_id = sample_point.id
    original_date_crea = sample_point.date_crea

    data = {
        "id": 99999,
        "date_crea": "2000-01-01",
        "nom": "Update"
    }

    updated = update_point_eau_by_id(db_session, sample_point.id,
data)

    assert updated.id == original_id
    assert updated.date_crea == original_date_crea
    assert updated.nom == "Update"
```

Test d'intégration : flux CRUD complet

Ce test valide que toutes les méthodes du DAO communiquent parfaitement entre elles pour un même objet, simulant une gestion complète en base de données.

```
def test_integration_full_crud(self, db_session):
    # CREATE
    data = {
        "numero_pei": 88888,
        "nom": "Point Integration",
        "statut": "public",
        "type_nature": "pi150",
        "latitude": 43.2965,
        "longitude": 5.3698,
        "press_deb": 5.0,
        "debit_1_bar": 60.0
    }
    created = create_point_eau(db_session, data)
    assert created.id is not None

    retrieved_by_id = get_point_eau_by_id(db_session, created.id)
    assert retrieved_by_id["numero_pei"] == 88888

    retrieved_by_numero = get_point_eau_by_numero_pei(db_session,
88888)
    assert retrieved_by_numero["id"] == created.id

    all_points = get_all_points_eau(db_session)
    assert any(p["id"] == created.id for p in all_points)

    update_data = {"nom": "Point Mis à Jour", "statut": "PRIVE"}
    updated = update_point_eau_by_id(db_session, created.id,
update_data)
    assert updated.nom == "Point Mis à Jour"
    assert updated.statut == "PRIVE"

    deleted = delete_point_eau_by_id(db_session, created.id)
    assert deleted is True
```

```
not_found = get_point_eau_by_id(db_session, created.id)
assert not_found is None
```

Test sécurité : Unicité du PEI

La sécurité repose ici sur l'impossibilité de créer des conflits d'identification.

```
def test_create_point_eau_numero_pei_unique(self, db_session,
sample_point):
    data = {
        "numero_pei": sample_point.numero_pei,
        "statut": "public",
        "type_nature": "bi",
        "latitude": 48.0,
        "longitude": 2.0
    }

    with pytest.raises(ValueError, match="existe déjà"):
        create_point_eau(db_session, data)
```

Modules Missions :

Les tests DAO garantissent que chaque mission est rattachée à des entités valides (Utilisateurs et PEI) et que son cycle de vie est protégé contre les modifications incohérentes.

Justification :

- Tests unitaires : Ils valident les fonctions de base en isolation, comme la transformation automatique des objets SQLAlchemy en dictionnaires pour faciliter l'usage par l'API.
- Tests fonctionnels : Ils simulent des scénarios métiers réels, notamment le filtrage chronologique (missions du jour) et la gestion des commentaires de terrain.
- Tests d'intégration : Ils vérifient la cohérence du cycle de vie complet (Création → Mise à jour → Suppression) et la solidité des relations entre les tables.

- Tests de sécurité : Ils assurent l'intégrité référentielle en bloquant la création de missions liées à des utilisateurs ou des points d'eau inexistant, et protègent les champs système contre les altérations.

Illustration avec les tests principaux :

Test unitaire : Transformation et accès

Ce test vérifie que le DAO renvoie des données propres (dictionnaires) sans exposer les états internes de SQLAlchemy.

```
def test_get_all_mission_returns_dict(self, db_session, mission_test):  
    missions = get_all_mission(db_session)  
    assert isinstance(missions[0], dict)  
    assert "_sa_instance_state" not in missions[0]
```

Test fonctionnel : Filtrage métier

Ce test valide la capacité du système à isoler les missions selon leur date, une fonctionnalité critique pour l'organisation quotidienne des tournées de vérification.

```
def test_get_mission_by_date_today(self, db_session, mission_test):  
    today = date.today()  
    missions = get_mission_by_date(db_session, today)  
    assert len(missions) >= 1
```

Test d'intégration : Cycle de vie complet

Ce test assure qu'une mission peut traverser toutes les étapes de son existence en base de données sans perte d'information ni erreur de contrainte.

```
def test_crud_complete_lifecycle(self, db_session, utilisateur_test, point_test):  
    data = {  
        "nom_mission": "Mission CRUD",  
        "id_point": point_test.numero_pei,
```

```

        "id_utilisateur": utilisateur_test.id_utilisateur
    }
    created = create_mission(db_session, data)

    found = get_mission_by_id(db_session, created.id_mission)
    assert found.nom_mission == "Mission CRUD"

    updated = update_mission_by_id(db_session, created.id_mission,
{"statut": "TERMINER"})
    assert updated.statut == "TERMINER"

    deleted = delete_mission_by_id(db_session, created.id_mission)
    assert deleted is True
    assert get_mission_by_id(db_session, created.id_mission) is
None

```

Test sécurité : Validation des clés étrangères

La sécurité du système repose sur l'impossibilité de créer des données orphelines. Ce test vérifie que le DAO rejette toute mission liée à un point d'eau non répertorié.

```

def test_create_mission_invalid_point(self, db_session,
utilisateur_test):
    data = {
        "nom_mission": "Mission Invalide",
        "id_point": 99999,
        "id_utilisateur": utilisateur_test.id_utilisateur
    }

    with pytest.raises(ValueError, match="id du point est
invalide"):
        create_mission(db_session, data)

```

Module Historique :

nos tests du DAO garantissent que chaque action administrative ou opérationnelle laisse une trace immuable en base de données, permettant ainsi un suivi précis des modifications et des connexions.

Justification :

- Tests unitaires : Ils valident la structure de données renvoyée. Il est crucial que les fonctions de listing renvoient des dictionnaires propres pour l'API, tandis que les recherches par ID doivent renvoyer des objets ORM complets.
- Tests fonctionnels : Ils vérifient la logique métier de l'audit, notamment le tri décroissant.
- Tests d'intégration : Ils simulent un flux d'activité utilisateur complet (Connexion → Action → Déconnexion) pour valider que la base de données maintient l'ordre et la cohérence des logs sur une session.
- Tests de sécurité : Ils traitent deux risques majeurs : l'intégrité référentielle et la protection contre les injections SQL via des champs textuels comme l'adresse IP.

Illustration avec les tests principaux :

Test unitaire : intégrité des types de retour

Ce test vérifie que le DAO convertit correctement les données SQL en formats exploitables par l'application.

```
def test_get_all_historique_returns_list_of_dicts(self, db_session,
historique_test):
    logs = get_all_historique(db_session)
    assert len(logs) >= 1
    assert isinstance(logs[0], dict)
    assert logs[0]["action"] == "CONNEXION"
```

Test fonctionnel : Tri chronologique

Le système doit garantir que la méthode de récupération des dernières actions renvoie toujours aux événements les plus récents en premier.

```
def test_get_derniere_action_sorting(self, db_session,
utilisateur_test):
    """Vérifie que les actions sont bien triées de la plus récente
à la plus ancienne"""
    # Création de deux logs à la suite
```

```

        create_historique(db_session, {"id_utilisateur":
utilisateur_test.id_utilisateur, "action": "FIRST", "cible": "T1"})
        create_historique(db_session, {"id_utilisateur":
utilisateur_test.id_utilisateur, "action": "SECOND", "cible": "T2"})

    recent_logs = get_derniere_action(db_session, limit=2)
    assert recent_logs[0].action == "SECOND"
    assert recent_logs[1].action == "FIRST"

```

Test d'intégration : Flux d'activité

On vérifie ici que le DAO est capable de reconstituer fidèlement le parcours d'un utilisateur sans mélange de données.

```

def test_user_activity_flow(self, db_session, utilisateur_test):
    # Simuler une série d'actions utilisateur
    actions = ["LOGIN", "VIEW_MAP", "LOGOUT"]
    for act in actions:
        create_historique(db_session, {
            "id_utilisateur": utilisateur_test.id_utilisateur,
            "action": act,
            "cible": "SYSTEM"
        })

    # Récupérer tout l'historique de cet utilisateur
    user_history = get_historique_by_utilisateur(db_session,
utilisateur_test.id_utilisateur)

    assert len(user_history) == 3
    # Vérification de l'ordre décroissant
    assert user_history[0].action == "LOGOUT"
    assert user_history[-1].action == "LOGIN"

```

Test sécurité : Résistance aux injections SQL

Ce test utilise une charge utile malveillante dans le champ IP pour confirmer que l'ORM (SQLAlchemy) neutralise correctement les tentatives de destruction de table.

```

def test_sql_injection_on_ip_field(self, db_session,

```

```

utilisateur_test):
    """Vérifie que les entrées malveillantes sont traitées comme
    du texte (via ORM)"""
    malicious_ip = "127.0.0.1"; DROP TABLE historiques; --"
    data = {
        "id_utilisateur": utilisateur_test.id_utilisateur,
        "action": "ATTACK",
        "cible": "SERVER",
        "ip": malicious_ip
    }
    log = create_historique(db_session, data)

    assert log.ip == malicious_ip
    # Vérifier que la table existe toujours
    assert get_all_historique(db_session) is not None

```

Module Signaler :

Les tests DAO garantissent que les signalements sont correctement rattachés aux PEI et que les fichiers associés (photos) sont gérés de manière sécurisée.

Justification :

- Tests unitaires : Ils valident la structure des dictionnaires retournés par le DAO. Nous vérifions que tous les champs nécessaires (ID, problème, photo, date) sont présents pour une exploitation correcte par le frontend.
- Tests fonctionnels : Ils simulent des cas d'usage réels, comme la mise à jour d'un signalement ou la possibilité pour un même point d'eau de cumuler plusieurs signalements distincts.
- Tests d'intégration : Ils vérifient le cycle de vie complet, incluant un aspect critique : la suppression des fichiers physiques. Si un signalement est supprimé en base, le DAO doit s'assurer que l'image stockée sur le serveur est également effacée.
- Tests de sécurité : Ils assurent l'intégrité du système en vérifiant les relations. Un signalement ne peut être créé que s'il est lié à un id_point (PEI) et à un id_utilisateur (agent) existants.

Illustration avec les tests principaux :

Test unitaire : validation du schéma de sortie

Ce test garantit que les données extraites de la base de données respectent le contrat d'interface attendu par les services supérieurs.

```
def test_get_all_signale_contains_expected_fields(self, db_session,
signalement_test):
    """Test que les signalements contiennent tous les champs
attendus"""
    signalements = get_all_signale(db_session)

    sig = signalements[0]
    assert "id" in sig
    assert "id_point" in sig
    assert "probleme" in sig
    assert "photo" in sig
    assert "id_utilisateur" in sig
    assert "date_creation" in sig
```

Test fonctionnel : Accumulation de signalement

Sur le terrain, un PEI peut avoir plusieurs problèmes. Ce test valide que le DAO gère correctement la multiplicité des signalements pour un seul identifiant de point.

```
def test_get_signale_by_id_point_multiple(self, db_session,
point_test, utilisateur_test):
    """Test plusieurs signalements pour le même point"""
    for i in range(3):
        sig = models.Signaler(
            id_point=point_test.numero_pei,
            probleme=f"Probleme {i}",
            photo=f"photo_{i}.jpg",
            id_utilisateur=utilisateur_test.id_utilisateur
        )
        db_session.add(sig)
        db_session.commit()

    signalements = get_signale_by_id_point(db_session,
point_test.numero_pei)
    assert len(signalements) == 3
    assert all(s["id_point"] == point_test.numero_pei for s in
signalements)
```

Test d'intégrité : Nettoyage du stockage (Photo)

C'est un test de robustesse : nous créons un fichier temporaire réel et vérifions que la suppression du signalement dans le DAO entraîne bien la suppression du fichier sur le disque.

```
def test_delete_signale_with_real_photo(self, db_session, point_test,
utilisateur_test):
    """Test suppression avec fichier photo réel"""
    # Créer un fichier temporaire
    with tempfile.NamedTemporaryFile(mode='w', delete=False,
suffix='.jpg') as f:
        temp_photo = f.name
        f.write("test photo content")

    try:
        # Créer signalement avec photo réelle
        sig = models.Signaler(
            id_point=point_test.numero_pei,
            probleme="Probleme avec photo",
            photo=temp_photo,
            id_utilisateur=utilisateur_test.id_utilisateur
        )
        db_session.add(sig)
        db_session.commit()

        # Vérifier que le fichier existe
        assert os.path.exists(temp_photo)

        # Supprimer le signalement
        result = delete_signale_by_id_point(db_session,
point_test.numero_pei)
        assert result is True

        # Vérifier que le fichier a été supprimé
        assert not os.path.exists(temp_photo)

    finally:
        # Nettoyage au cas où
        if os.path.exists(temp_photo):
```

```
os.remove(temp_photo)
```

Test sécurité : Intégrité référentielle

Ce test vérifie que le DAO rejette toute tentative de création de signalement orphelin (lié à un utilisateur ou un point d'eau fictif).

```
def test_create_signale_invalid_user(self, db_session, point_test):
    """Test création avec un id_utilisateur invalide"""
    signale_data = {
        "id_point": point_test.numero_pei,
        "probleme": "Probleme",
        "photo": None,
        "id_utilisateur": 99999
    }

    with pytest.raises(ValueError, match="id_utilisateur est incorrect"):
        create_signale(db_session, signale_data)
```

Test des routes API

Autant que la DAO , les Routes sont indispensables. Tester les routes permet de garantir que les contrats d'interface (JSON) sont respectés, que les codes HTTP (200, 401, 404, 422) sont corrects et que la sécurité par JWT est impénétrable.

Module Utilisateurs :

Les tests s'appuient sur FastAPI et TestClient pour simuler des appels réseau et vérifier que la couche de sécurité (JWT + RBAC) filtre correctement les accès selon les rôles

Justification :

- Tests d'authentification : Ils valident le cycle de vie des identifiants (Inscription → Connexion → Déconnexion). On s'assure que le serveur délivre bien un token JWT valide après chaque authentification réussie.
- Tests de gestion personnelle (/me) : Ils vérifient qu'un utilisateur peut consulter et modifier ses propres informations, tout en utilisant des dependency overrides pour simuler l'injection du current_user.

- Tests d'administration (RBAC) : Ils constituent le cœur de la sécurité. Nous testons explicitement le rejet des accès (Code HTTP 403) lorsqu'un utilisateur standard tente d'accéder à des routes réservées aux administrateurs.
- Tests de protections métier : Ils valident les garde-fous implémentés dans le router, notamment l'impossibilité pour un administrateur de supprimer son propre compte via la route administrative.

Illustration avec les tests principaux :

Test d'Authentification : Création et Token

Ce test vérifie que la route d'inscription traite correctement le payload et retourne les informations essentielles à la session utilisateur.

```
def test_create_user_success(self, client, user_payload):
    res = client.post("/utilisateurs/", json=user_payload)
    assert res.status_code == 200
    data = res.json()
    assert "id_utilisateur" in data
    assert "token" in data
```

Test de Profil : Injection de Dépendance

Ici, on utilise `dependency_overrides` pour simuler une session active sans avoir à repasser par le login, permettant de tester la route `/me` de manière isolée et rapide.

```
def test_get_my_profile_success(self, client, db_user):
    app.dependency_overrides[getTokenUser] = lambda: db_user
    res = client.get("/utilisateurs/me")

    assert res.status_code == 200
    assert res.json()["email"] == db_user.email
    app.dependency_overrides.clear()
```

Test de Sécurité : Contrôle des Rôles (RBAC)

Ce test est crucial pour la confidentialité des données : il garantit qu'un utilisateur public ne peut pas lister les autres membres du personnel.

```
def test_list_users_forbidden_for_user(self, client, db_user):
    app.dependency_overrides[getTokenUser] = lambda: db_user
    res = client.get("/utilisateurs/")

    assert res.status_code == 403
    app.dependency_overrides.clear()
```

Test de Protection : Auto-suppression interdite

Ce test valide la règle de sécurité empêchant un administrateur de se supprimer lui-même accidentellement, ce qui prévient la perte d'accès au système.

```
def test_delete_admin_self_forbidden(self, client, db_admin):
    app.dependency_overrides[getTokenUser] = lambda: db_admin
    res =
client.delete(f"/utilisateurs/{db_admin.id_utilisateur}")

    assert res.status_code == 400
    app.dependency_overrides.clear()
```

Routes Point d'eau :

Les tests des routes garantissent non seulement l'accès aux données techniques, mais aussi la validité de la projection géospatiale effectuée lors des appels API.

Justification :

- Tests de Listing et Recherche : Ils vérifient que les points d'eau sont correctement exposés au format JSON et que la recherche par identifiant métier (numero_pei) renvoie les bons objets techniques.
- Tests de Création: Contrairement aux utilisateurs, un PEI possède de nombreux champs techniques (débit, pression). Le test vérifie que le validateur Pydantic accepte ces données et que le backend génère la géométrie PostGIS associée.
- Tests d'Autorisation (RBAC) : La modification du patrimoine hydraulique est une action critique. Nous testons que seules les personnes authentifiées avec un rôle adéquat peuvent créer ou supprimer des points.

- Tests de Gestion d'Erreur : Validation des codes 404 Not Found pour les identifiants inexistantes et des codes 401/403 pour les tentatives d'accès non autorisées.

Illustration avec les tests principaux :

Test de Consultation : Récupération par Numéro PEI

Ce test valide que l'API est capable de retrouver une ressource via son identifiant métier unique.

```
def test_get_point_by_numero_success(self, client, point_eau_test):
    response =
client.get(f"/points-eau/{point_eau_test.numero_pei}")
    assert response.status_code == 200
    assert response.json()["numero_pei"] ==
point_eau_test.numero_pei
```

Test de Création : Validation et Géolocalisation

Ici, on teste l'envoi d'un payload complet. Le succès du test confirme que le système a bien transformé les coordonnées latitude/longitude envoyées par le client en un objet géométrique interne.

```
def test_create_point_success(self, client, valid_point_payload,
db_admin):
    # On simule la connexion d'un admin
    app.dependency_overrides[getTokenUser] = lambda: db_admin

    response = client.post("/points-eau/",
json=valid_point_payload)
    assert response.status_code == 200
    data = response.json()
    assert data['numero_pei'] == valid_point_payload['numero_pei']
    assert 'latitude' in data
```

Test de Sécurité : Protection des données

Ce test garantit qu'un utilisateur non authentifié (ou "anonyme") ne peut pas injecter de nouveaux points d'eau dans le réseau.

```
def test_create_point_forbidden_for_anonymous(self, client,
valid_point_payload):
    response = client.post("/points-eau/",
json=valid_point_payload)
    assert response.status_code in [401, 403, 422]
```

Test de Suppression : Nettoyage administratif

On vérifie que la route de suppression fonctionne correctement lorsqu'elle est sollicitée par un administrateur.

```
def test_delete_point_as_admin(self, client, point_eau_test,
db_admin):
    app.dependency_overrides[getTokenUser] = lambda: db_admin

    response =
client.delete(f"/points-eau/{point_eau_test.numero_pei}")
    assert response.status_code == 200
    assert "supprimé" in response.json()["detail"].lower()
```

Routes Missions :

Nos tests des routes valident l'attribution des tâches, le suivi des statuts (EN COURS, TERMINER) et la cohérence des relations entre les intervenants (pompiers) et les points d'eau (PEI).

Justification :

- Tests de Cycle de Vie : Ils vérifient qu'une mission peut franchir toutes les étapes, de sa création par un administrateur à sa clôture par un agent.
- Tests de Liaison Relationnelle : Contrairement aux autres modules, une mission est un pivot. Le test valide que l'API rejette toute mission qui ne serait pas liée à la fois à un PEI existant et à un utilisateur valide.
- Tests de Mise à Jour de Statut : Le changement de statut est l'action la plus fréquente. Nous testons que le Payload de mise à jour (PUT) modifie correctement l'état de la mission sans altérer les données d'origine (date de création, etc.).

- Tests d'Autorisation (RBAC) : On s'assure que la création et la suppression de missions sont des privilèges administratifs, tandis que la consultation peut être ouverte aux agents (pompiers).

Illustration avec les tests principaux :

Test de Création : Attribution de tâche

Ce test simule l'ordre de mission donné par un administrateur. Il valide que le système accepte les commentaires et prépare l'itinéraire pour l'agent.

```
def test_create_mission_success(self, client, db_admin, db_pompier,
point_eau_test):
    app.dependency_overrides[getTokenUser] = lambda: db_admin
    payload = {
        "nom_mission": "Verification Borne",
        "id_point": point_eau_test.numero_pei,
        "id_utilisateur": db_pompier.id_utilisateur,
        "commentaire": "A vérifier d'urgence",
        "itineraire": None
    }
    response = client.post("/missions/", json=payload)
    assert response.status_code == 200
    app.dependency_overrides.clear()
```

Test de Mise à Jour : Clôture de Mission

Ce test vérifie la capacité du système à enregistrer la fin d'une intervention. C'est ici que l'on valide le passage au statut "TERMINER".

```
def test_update_mission_success(self, client, mission_test,
db_admin):
    app.dependency_overrides[getTokenUser] = lambda: db_admin

    payload = {
        "nom_mission": "Mission modifiée",
        "id_point": mission_test.id_point,
        "id_utilisateur": mission_test.id_utilisateur,
        "statut": "TERMINER",
        "commentaire": "Terminé par le test",
```

```

        "itineraire": None,
        "date_fin": None
    }
    response =
client.put(f"/missions/update/{mission_test.id_mission}",
json=payload)
    if response.status_code == 422:
        print(f"\nERREUR VALDIATION : {response.json()}")

    assert response.status_code == 200
    assert response.json()['statut'] == "TERMINER"
    app.dependency_overrides.clear()

```

Test de Consultation : Liste de travail

On vérifie ici qu'un agent (rôle pompier) peut accéder à la liste des missions qui lui sont confiées via la route globale.

```

def test_get_all_missions(self, client, mission_test, db_pompier):
    app.dependency_overrides[getTokenUser] = lambda: db_pompier
    response = client.get("/missions/")
    assert response.status_code == 200
    assert len(response.json()) >= 1
    app.dependency_overrides.clear()

```

Test de Suppression : Sécurité administrative

Ce test confirme que les missions obsolètes ou créées par erreur peuvent être retirées du système par les gestionnaires.

```

def test_delete_mission_success(self, client, mission_test,
db_admin):
    app.dependency_overrides[getTokenUser] = lambda: db_admin
    response =
client.delete(f"/missions/supprimer/{mission_test.id_mission}")
    assert response.status_code == 200
    app.dependency_overrides.clear()

```

Routes Historique :

Le module Historique est le système de surveillance de l'application. Contrairement aux autres modules, son accès est strictement hiérarchisé.

Justification :

- Tests de visibilité hiérarchique : Nous vérifions que les journaux d'audit sont accessibles aux officiers (commandement) et aux pompiers pour la traçabilité, mais totalement masqués au rôle public (Code 403).
- Tests de filtrage par utilisateur : Une route spécifique permet de reconstruire le parcours d'un agent précis. Le test garantit que l'API filtre correctement les données sans mélange entre les comptes.
- Tests de limitation : L'historique pouvant contenir des milliers de lignes, nous testons le paramètre limit pour s'assurer que l'API ne surcharge pas le réseau et respecte les contraintes de performance.
- Tests de restriction d'écriture : L'audit doit être infalsifiable. Nous testons explicitement que seul un admin peut poster une nouvelle entrée, bloquant ainsi toute tentative de manipulation des logs par un rôle inférieur.

Illustration avec les tests principaux :

Test de Sécurité : Accès Interdit au Public

C'est le garde-fou principal : les données de connexion et d'actions techniques ne doivent jamais fuir vers les utilisateurs externes.

```
def test_get_all_historique_forbidden_for_public(client,
user_public):
    app.dependency_overrides[getTokenUser] = lambda: user_public

    response = client.get("/historique/")
    assert response.status_code == 403
```

Test de Consultation : Audit d'un Utilisateur

Ce test valide la capacité du centre de commandement à surveiller les actions d'un agent spécifique sur le terrain.

```
def test_get_historique_by_utilisateur(client, db_session, user_cmd,
user_public):

db_session.add(models.Historique(id_utilisateur=user_public.id_utilis
ateur, action="B", cible="T", ip="2"))
db_session.commit()

app.dependency_overrides[getTokenUser] = lambda: user_cmd

response =
client.get(f"/historique/utilisateur/{user_public.id_utilisateur}")
assert response.status_code == 200
assert response.json()[0]["id_utilisateur"] ==
user_public.id_utilisateur
```

Test de Performance : Limitation des résultats

On vérifie ici que le paramètre de requête limite est bien pris en compte par le backend pour optimiser l'affichage des dernières actions.

```
def test_get_dernieres_actions_limit(client, db_session, user_cmd,
user_public):
    for i in range(3):

db_session.add(models.Historique(id_utilisateur=user_public.id_utilis
ateur, action=f"Act{i}", cible="T", ip="0"))
db_session.commit()

app.dependency_overrides[getTokenUser] = lambda: user_cmd

response = client.get("/historique/derniere-actions?limit=2")
assert response.status_code == 200
assert len(response.json()) == 2
```

Test de Privilège : Création d'Audit

Seul l'administrateur système doit pouvoir notifier manuellement un événement dans l'historique via l'API.

```
def test_create_historique_as_admin(client, user_admin, user_public):
    app.dependency_overrides[getTokenUser] = lambda: user_admin

    payload = {
        "id_utilisateur": user_public.id_utilisateur,
        "action": "Creation",
        "cible": "PEI",
        "ip": "10.0.0.1"
    }
    response = client.post("/historique/", json=payload)
    assert response.status_code == 200
    assert response.json()["action"] == "Creation"
```

Route Signaler :

Le module Signalements permet la remontée d'incidents terrain. Contrairement aux autres routes, il gère des flux mixtes. Les tests valident la réception des images, leur stockage et le lien logique avec les Points d'Eau (PEI).

Justification :

- Tests de Flux Multipart : C'est le point critique. Nous utilisons io.BytesIO pour simuler l'envoi d'un fichier image réel sans encombrer le disque, vérifiant que l'API traite correctement les objets UploadFile de FastAPI.
- Tests de Liaison et Intégrité : Un signalement est inutile s'il n'est pas rattaché à un PEI existant. Le test vérifie que l'API retourne un code 404 Not Found si l'identifiant du point est erroné.
- Tests d'Autorisation (RBAC) : Si tout utilisateur authentifié peut signaler un problème, la suppression des signalements est une action sensible réservée aux administrateurs pour éviter l'effacement de preuves d'incidents.
- Tests de Suppression Physique : Validation de la route de nettoyage qui doit faire le lien entre la base de données et l'identifiant du point pour purger les alertes résolues.

Illustration avec les tests principaux :

Test de Création : Upload de photo et Signalement

Ce test simule un agent sur le terrain envoyant une photo d'un sinistre. Il valide que le serveur accepte le format Multipart/Form-Data

```

def test_create_signalement_success(self, client, db_session,
db_user):
    numero_pei = random.randint(100000, 999999)
    point = models.PointEau(
        numero_pei=numero_pei, nom="Point", statut="PUBLIC",
type_nature="BI",
        geom=WKTElement('POINT(200000 6800000)', srid=2154)
    )
    db_session.add(point)
    db_session.commit()

    payload = {
        "id_point": str(numero_pei),
        "probleme": "Bouche incendie cassée",
        "id_utilisateur": str(db_user.id_utilisateur)
    }

    file_data = {"photo": ("test.jpg",
io.BytesIO(b"fake-image-content"), "image/jpeg")}

    response = client.post("/signaler/", data=payload,
files=file_data)

    assert response.status_code == 200
    assert response.json()["id_point"] == numero_pei

```

Test de Gestion d'Erreur : Point Inexistant

L'API doit être robuste et empêcher la création de signalements "orphelins" qui ne pourraient pas être localisés sur la carte.

```

def test_create_signalement_point_not_found(self, client, db_user):
    payload = {
        "id_point": "99999999",
        "probleme": "Inconnu",
        "id_utilisateur": str(db_user.id_utilisateur)
    }

    file_data = {"photo": ("test.jpg", io.BytesIO(b"img"),
"image/jpeg")}

```

```
response = client.post("/signaler/", data=payload,
files=file_data)
assert response.status_code == 404
assert "n'a pas été trouvé" in response.json()["detail"]
```

Test de Consultation : Flux d'incidents

On vérifie que la liste globale des signalements est accessible aux agents pour qu'ils puissent prendre connaissance des pannes déclarées.

```
def test_get_all_signalements(self, client, signalement_test,
db_user):
    app.dependency_overrides[getTokenUser] = lambda: db_user

    response = client.get("/signaler/")
    assert response.status_code == 200
    assert len(response.json()) >= 1
    app.dependency_overrides.clear()
```

Test de Suppression : Sécurité Admin

Seul un administrateur peut valider la résolution d'un problème et supprimer le signalement associé.

```
def test_delete_signalement_as_admin(self, client, signalement_test,
db_session):
    admin = models.Utilisateur(
        nom="Admin",
        prenom="System",
        role=RoleEnum.admin,
        email=f"adm{random.randint(1,999)}@test.com",
        telephone="0611223344",
        mot_de_passe="h"
    )
    db_session.add(admin)
    db_session.commit()

    app.dependency_overrides[getTokenUser] = lambda: admin
```

```
response =  
client.delete(f"/signaler/supprimer/{signalement_test.id_point}")  
assert response.status_code == 200  
app.dependency_overrides.clear()
```

Conclusion

Les tests restent encore désorganiser mes touches les actions et points principaux de l'application permettent d'avoir un vision globale des fonctionnalités, de leur fiabilité et des possibilités d'évolution futur pour optimiser notre code.